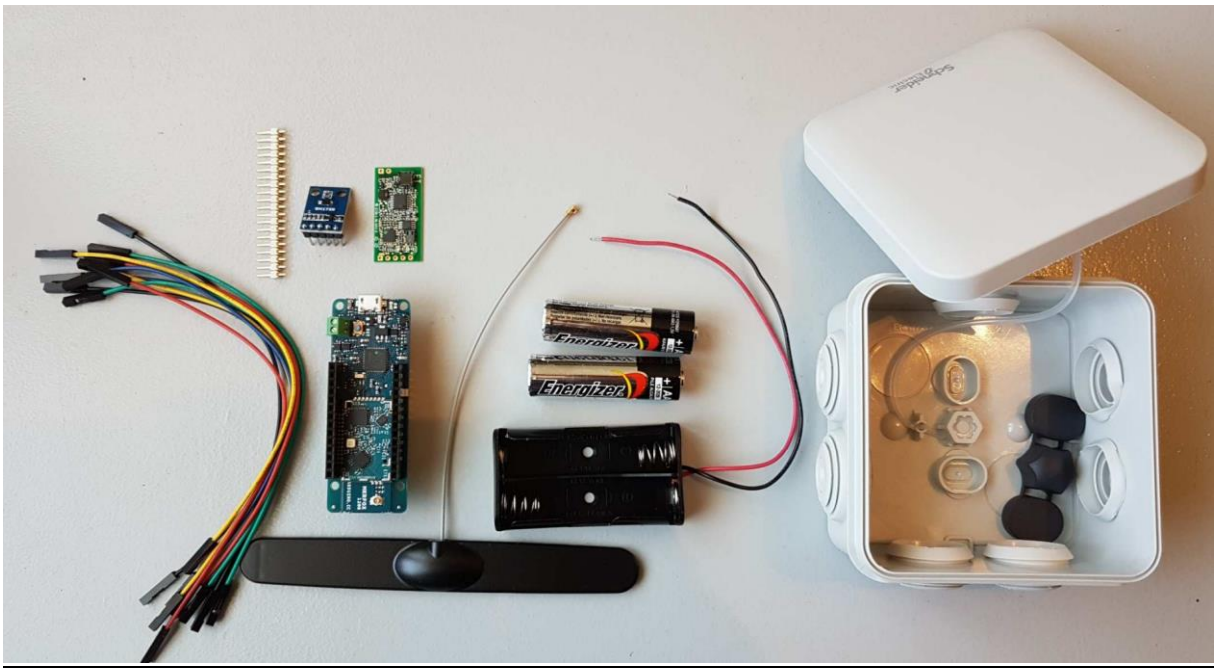
MC43



Thème 3 : Mise en œuvre d'un objet connecté par réseau Sigfox

Réalisation d'une station météo

Réalisé par INZOUDDINE Monia, RAMIN Ronan

Encadré par DEPERNET Daniel

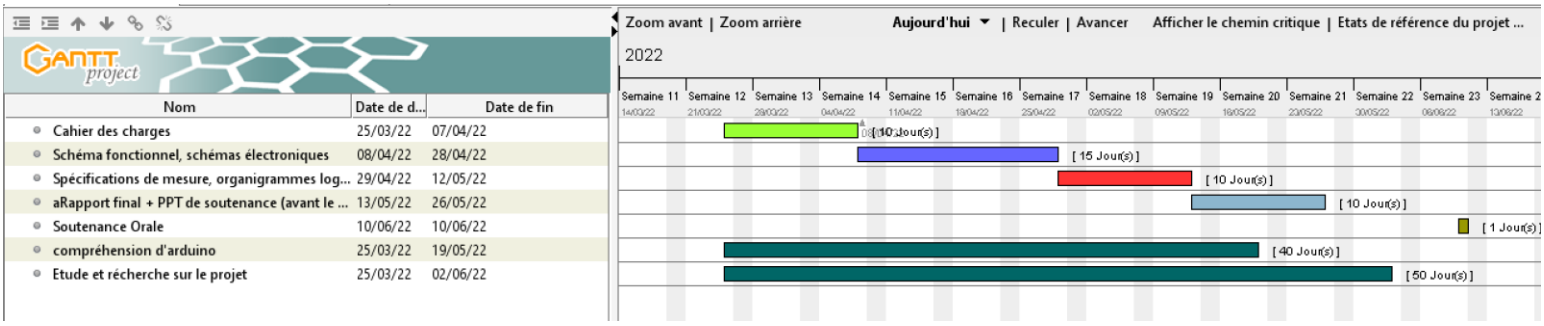
10/06 /2022 à Belfort

Table des matières

I. Cahier des charges 3
A. Schéma de gantt : 3
B. Objectif : 3
C. Description générale : 3
D. Description des caractéristiques principales des produits utilisés : 3
II. Schéma fonctionnel 7
III. Schémas électroniques 9
IV. Spécifications de mesure 11
a. Spécifications de mesure du BME280 11
b. Spécifications des mesures du capteur de pluie 13
c. Spécifications de mesure du capteur de luminosité BH1750 : 13
d. Spécification de mesure du capteur de distance SFR02 14
V. Organigrammes logiciels 15
VI. Expérimentations et tests : 17
Test BME280 : 17
Test le capteur de pluie : 21
Test du capteur de distance SFR02 : 22
Test du capteur de luminosité BH1750 : 23
VII. Envoie des données sur Sigfox 24
VIII. ThingSpeak 29
IX. Conclusion : 32
X. Annexe 33

I. Cahier des charges

A. Schéma de gantt :



B. Objectif :

Conception et interprétation des données d'une station météo sur l'environnement Arduino par communication Sigfox.

C. Description générale :

Pour cela, on utilisera **un capteur de pluie** et **un capteur de température, un capteur d'humidité et un capteur de pression combinés**. On ajoutera également un capteur de luminosité le BH1750 et un capteur de distance avec ultrason.

Environnement de travail :

- Sigfox est un système de connexion utilisant des signaux de fréquence radio ultra-rapide et de longues portées, appelle la bande ultra étroite (UNB = Ultra Narrow Band).

Peu énergivore, elle utilise des bandes de fréquence libre de droit disponible pour le monde entier, comme les bandes ISM (Bande industrielle, scientifique et médicale). En Europe, il s'agit de l'ISL à 868 MHz.

Ce signal peut atteindre des zones reculées.

Sur ce réseau, les objets peuvent recevoir jusqu'à 140 messages de 12 octets par jour.

- On utilisera un microcontrôleur Arduino.

On utilisera la carte Arduino MKRFOX 1200, le capteur BME280 ainsi que le capteur de pluie.

D. Description des caractéristiques principales des produits utilisés :

Caractéristique principale du composant MKR1200 :

Le microcontrôleur a une tension d'entrée de 3,3V et à une interface de communication en I2C.

Microcontroller	SAMD21 Cortex-M0+ 32bit low power ARM MCU
Board Power Supply (USB/VIN)	5V
Supported Batteries(*)	2x AA or AAA
Circuit Operating Voltage	3.3V
Digital I/O Pins	8
PWM Pins	12 (0, 1, 2, 3, 4, 5, 6, 7, 8, 10, A3 - or 18 -, A4 -or 19)
UART	1
SPI	1
I2C	1
Analog Input Pins	7 (ADC 8/10/12 bit)
Analog Output Pins	1 (DAC 10 bit)

Caractéristique principale du composant BME280 :

Il a une interface I2C, il peut fonctionner avec un gramme en tension de 1,71 V à 3,6V, les courants consommés son très faibles de l'ordre du 10^{-6}. Et sa gamme de température, pression et humidité sont larges pour les mesures qu'on veut effectuer.

• Package	2.5 mm x 2.5 mm x 0.93 mm metal lid LGA
• Digital interface	I²C (up to 3.4 MHz) and SPI (3 and 4 wire, up to 10 MHz)
• Supply voltage	V_{DD} main supply voltage range: 1.71 V to 3.6 V V_{DDIO} interface voltage range: 1.2 V to 3.6 V
• Current consumption	1.8 µA @ 1 Hz humidity and temperature 2.8 µA @ 1 Hz pressure and temperature 3.6 µA @ 1 Hz humidity, pressure and temperature 0.1 µA in sleep mode
• Operating range	-40…+85 °C, 0…100 % rel. humidity, 300…1100 hPa
• Humidity sensor and pressure sensor can be independently enabled / disabled	

Avantages de ce produit :

- L'une des principales caractéristiques du BME280 est sa faible consommation d'énergie, ce qui rend l'appareil adapté à applications de récupération de batterie ou d'énergie, il permet la mise en œuvre des appareils alimentés par batterie tels que les modules GPS et les montres.
- **Il est donc parfaitement adapté à nos mesures.**
- Le capteur d'humidité et de pression offrent un temps de réponse extrêmement rapide pour les applications et une grande précision globale sur une large plage de températures.
- Le capteur de température a été optimisé pour une détection de bruit le plus faible et une résolution maximale.
- Le filtre IIR réduit efficacement la bande passante de la sortie de température et de pression et augmente la résolution des données de sortie de pression et de température à 20 bits.

Fonctionnement :

Le BME280 dispose de trois modes de fonctionnement : le mode veille, le mode normal et le mode forcé.

- Mode veille : pas de fonctionnement, tous les registres accessibles, puissance la plus faible, sélectionné après le démarrage
- Mode forcé : effectuer une mesure, stocker les résultats et revenir en mode veille
- Mode normal : cycle perpétuel des mesures et périodes d'inactivité.

Nous pensons utiliser le **mode forcé**.

Configuration :

Spécification des caractères d'humidité :

La mesure de l'humidité peut être activée ou ignoré. La résolution de la mesure de l'humidité est fixée à une sortie ADC 16 bits.

Spécification du capteur de pression :

La mesure de la pression peut être activée ou ignoré. La mesure de pression peut être lorsque le filtre IIR est activé, avoir une résolution de pression de 20 bits avec le filtre IIR. Lorsque le filtre IIR est désactivé, la résolution de pression est de 16 + (osrs_t – 1).

Spécification du capteur de température :

La mesure de la température peut être activée ou ignoré. Lorsque le filtre IIR est activé, la résolution de température est de 20 bits. Lorsque le filtre IIR est désactivé, la résolution de température est de 16 + (osrs_t – 1) bits.

Le capteur de pluie :

Le module de capteur de pluie est un outil pour la détection de la pluie. Il peut être utilisé comme un interrupteur lorsque la goutte de pluie tombe à travers la planche de pluie et aussi pour mesurer l'intensité des précipitations. Les caractéristiques du module, une planche de pluie et la carte de contrôle qui est séparée pour plus de commodité, voyant d'alimentation LED et une sensibilité réglable grâce à un potentiomètre.

Quelques fonctionnalités :

- Tension d'alimentation 5V
- Tension analogique de sortie A0

Matériels : récipient et eau

En ce qui concerne le capteur de température, d'humidité et de pression, on mesurera ces données dans la salle de cours pour vérifier le bon fonctionnement de notre application. Nous essaierons également de changer d'environnement pour voir l'impact sur les données et d'avoir des données justes.

Et pour le capteur de pluie, nous mesurerons pour différentes quantités d'eau l'impact sur les données. Si le temps le permet (s'il pleut on fera des mesures à l'extérieur)

Les données collectées seront ensuite transmises sur Sigfox via un protocole bien défini.

Caractéristique principale du composant BH1750 :

Le BH1750 est un capteur de lumière ambiante 16 bits qui communique via le protocole I2C. Il produit des mesures de luminosité en lux (unité d'éclairement dérivée du SI). Il peut mesurer un minimum de 1 lux et un maximum de 65535 lux.

Le but de ce composant est de détecté si nous sommes le jour où la nuit.

●Operating Conditions

Parameter	Symbol	Ratings			Units
		Min.	Typ.	Max.	
V_{CC} Voltage	Vcc	2.4	3.0	3.6	V
I^2C Reference Voltage	V_{DVI}	1.65	-	V_{CC}	V

Fonctionnement :

On utilisera High résolution mode l'affichage aura une précision de 1 lux.

Low Resolution Mode	4 lux precision	16 ms measurement time
High Resolution Mode	1 lux precision	120 ms measurement time
High Resolution Mode 2	0.5 lux precision	120 ms measurement time

---

<u>Caractéristiques principales du composant SFR02 :</u>

Ce capteur est un capteur de distance, il nous sert dans notre projet pour détecter les éventuels obstacles devant notre station météo en particulier si cela gêne le capteur de luminosité afin de ne pas fausser nos mesures.

Le principe est d'envoyer un signal (lumineux, radio ou sonore) et de mesurer en combien de temps ce signal revient à son point d'origine après avoir « rebondit » sur un obstacle. Lorsqu'on connait la vitesse de propagation de la lumière ou des ondes sonores, on peut déterminer à quelle distance se trouve un objet grâce à la formule connue de tous : vitesse = distance / temps, soit distance = vitesse x temps.

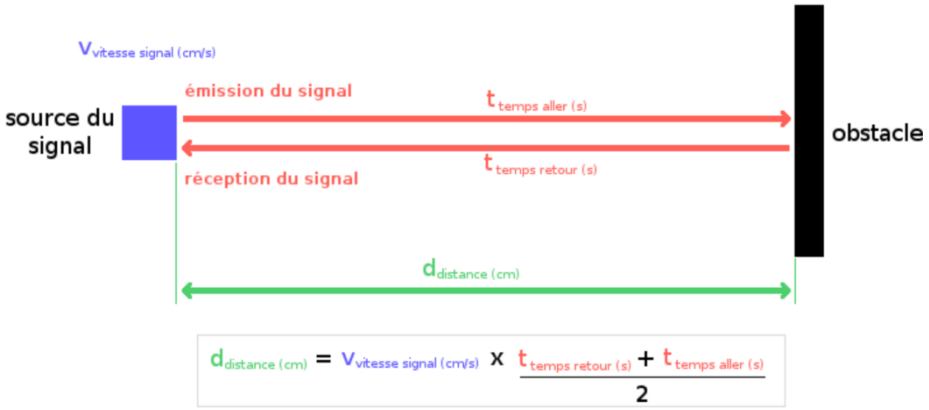
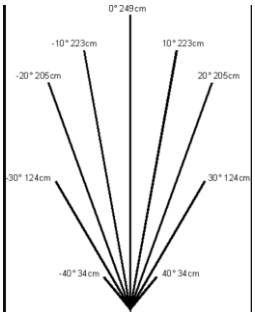


Schéma d'explication de la mesure de la distance

Ce module « autonome » est parfaitement adapté au Rapsberry Pi. Alimenté en 5V il ne consomme que 4mA et il dispose d'un bus I2C. Le SRF02 permet de mesurer des distances de 16cm à 6m avec une précision de l'ordre de 1 à 2 cm. L'ensemble est très petit (24x20x17 mm), et ne nécessite pas de calibration

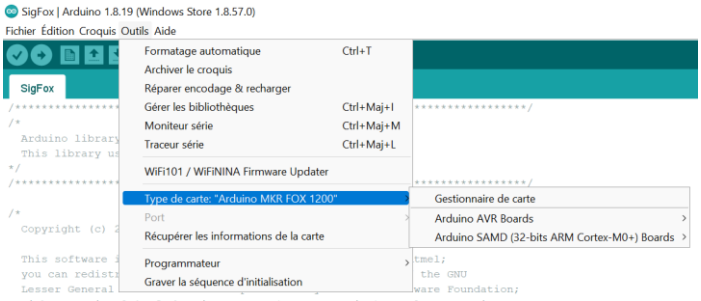
<u>Fonctionnement :</u>

Son cône de détection est assez large, cela nous convient pour notre application de-40 à +40°.



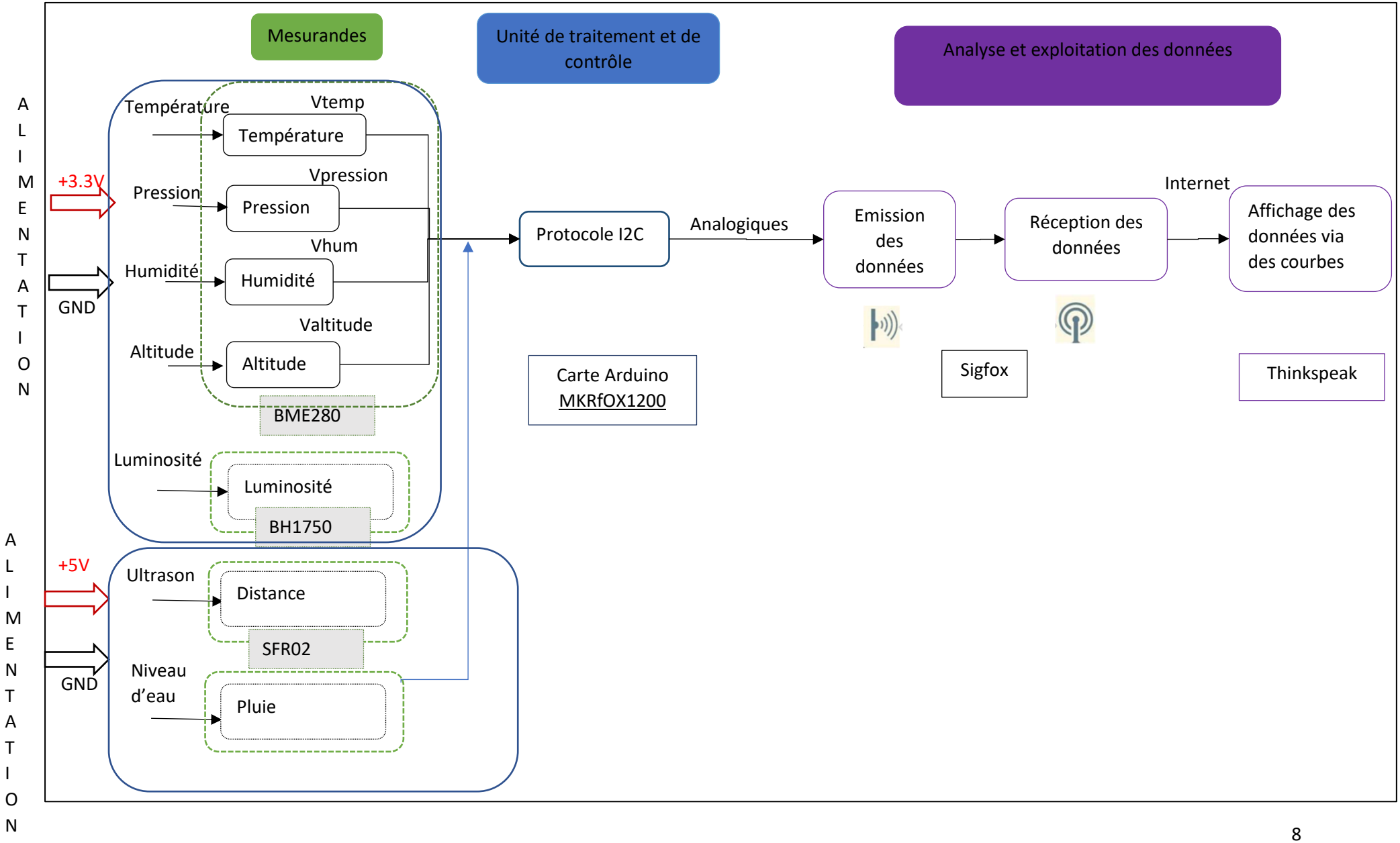
Arduino :

Il nous faudra au préalable télécharger les différentes bibliothèques souhaitées pour assurer le bon fonctionnement du projet. Pour cela nous allons consulter les différentes bibliothèques disponibles sur Moodle. Et également inclure la carte dans le répertoire outil de notre logiciel Arduino.

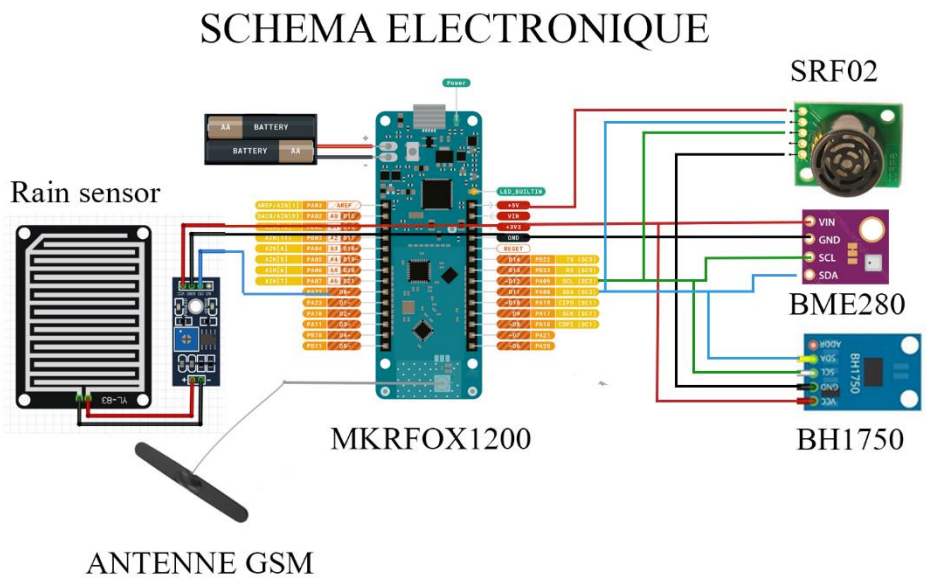


Dans la partie gestionnaire de tâches, nous avons ajouté la suite Arduino SAMD, capable de prendre en charge notre carte Arduino MKR FOX 1200. L'ajout de cette carte est essentiel au bon fonctionnement de notre projet car ce-dernier va nous permettre de téléverser notre code sur notre carte Arduino.

II. Schéma fonctionnel


Mesurandes
Unité de traitement et de contrôle
Analyse et exploitation des données
ALIMENTATION
+3.3V
GND
Température
Vtemp
Température
Pression
Vpression
Pression
Humidité
Vhum
Humidité
Altitude
Valtitude
Altitude
BME280
Luminosité
Luminosité
BH1750
Protocole I2C
Analogiques
Emission des données
Réception des données
Internet
Affichage des données via des courbes
Carte Arduino MKRfOX1200
Sigfox
Thinkspeak
ALIMENTATION
+5V
GND
Ultrason
Distance
SFR02
Niveau d'eau
Pluie

III. Schémas électroniques



Pour la partie électronique nous avons choisis d'utiliser le composant BME280. Comme indiquer dans sa datasheet, ce capteur va nous permettre de mesurer des températures entre -40°C et 85°C, de savoir également le taux d'humidité entre 0 et 100% RH, la pression entre 300hPa et 1100Pa ainsi que l'altitude entre 0 et 30 000ft.

De plus, nous avons choisi ce capteur que nous pouvons directement alimenter en +3.3V à partir de notre carte MKRFOX1200. Effectivement, ce-dernier peut être alimenter en +3.3V ou en +5V car il possède un module LM6206 qui est un régulateur +3.3V et également un traducteur de niveau de tension I2C (I2C Voltage Level Translator), nous servons donc à avoir une tension propre afin de transmettre plus facilement l'information.

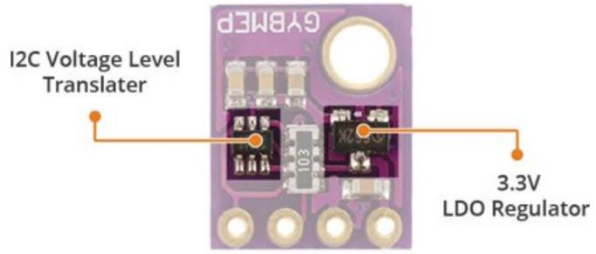


Schéma BME280 : régulateur et transmission de données par I2C

Autre aspect important de notre composant pour notre projet, sa consommation. En effet, le BME280 consomme moins d'un 1mA pendant la prise de mesures et seulement 5µA durant son temps de repos grâce à son sleep mode par défaut que nous pouvons contrôler à l'aide de la tension Vdd ou Vddio. En supplément, nous pouvons aussi utiliser le forced mode qui nous permet d'effectuer une seule période de mesure et de filtrer les options de mesures. Cela peut nous être très utile pour économiser de l'énergie ou bien encore demander une seule des mesures souhaité (ex : la température).

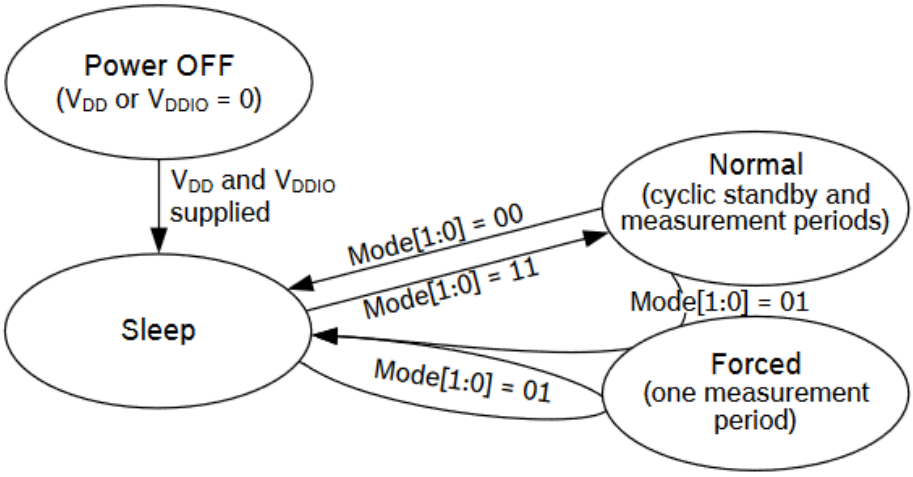


Schéma de l'utilisation des différents modes

IV. Spécifications de mesure

a. Spécifications de mesure du BME280

Table 1: Electrical parameter specification

Parameter	Symbol	Condition	Min	Typ	Max	Unit
Supply Voltage Internal Domains	V_{DD}	ripple max. 50 mVpp	1.71	1.8	3.6	V

On remarque que le BME280 a une tension d'entrée moyenne à 1.8 V et d'après la datasheet on une consommation de quelques dizaines de micro-ampères pour les mesures.

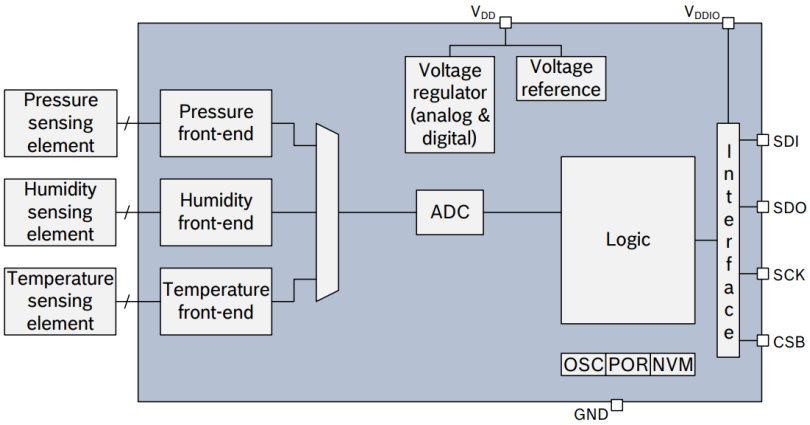


Figure 1 : Diagramme simplifié de mesure

Le diagramme ci-dessus explique comment la mesure va être réalisée et transférée au microcontrôleur. Le filtrage des mesures est fait avec un filtre passe bas du 1er ordre.

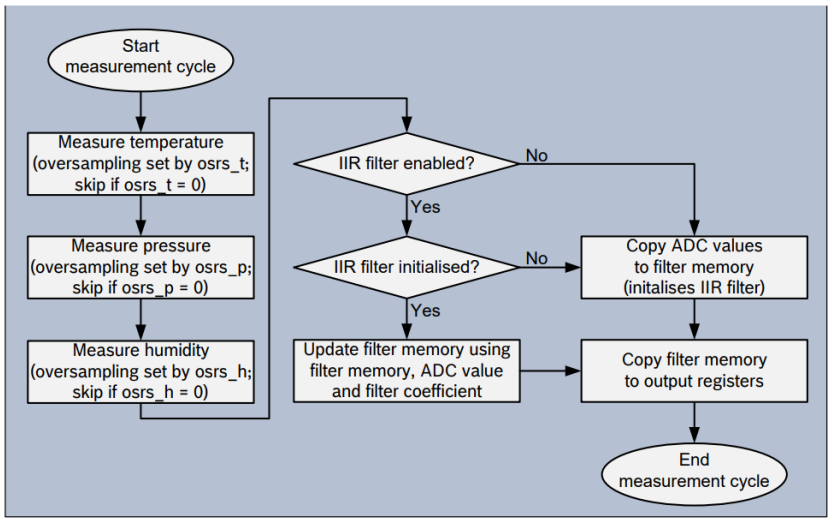


Figure 2 : Organigramme du principe de mesure

Les grandes étapes de la mesure sont :

1. mesure (avec option de suréchantillonnage)

2. ADC

3. Filtrage optionnel

Principe de mesure et de filtrage de la température, de la pression et de l'humidité :

Mesure de la température :

Parameter	Symbol	Condition	Min	Typ	Max	Unit
Operating range	T	Operational	-40	25	85	°C
		Full accuracy	0		65	°C

Les température peuvent être compris entre **-40 et 85 °C** .

Mesure de l'humidité :

Parameter	Symbol	Condition	Min	Typ	Max	Unit
Operating range[1]	R_H	For temperatures < 0 °C and > 60 °C see Figure 1	-40	25	85	°C
			0		100	%RH

Pour l'humidité, on remarque qu'elle sera en pourcentage elle est proportionnelle à la valeur de température mesurée.

Mesure de la pression :

Operating pressure range	P	full accuracy	300		1100	hPa

Mesure de l'altitude :

Carte de l'initialisation de l'altitude

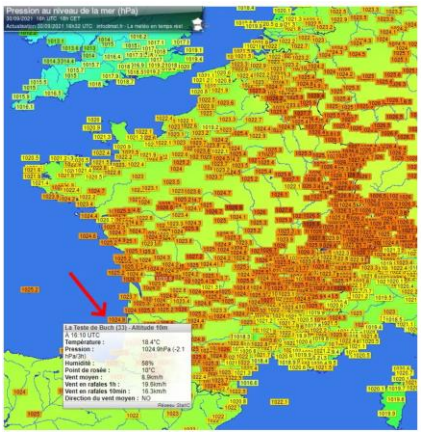


Figure 3 : Carte pour initialisation de l'altitude

Comme l'altitude est différent selon l'endroit où l'on se trouve, il est plus judicieux d'initialiser l'altitude pour ne pas avoir une erreur de plusieurs dizaines de mètres. La mesure de l'al

Gestion d'énergie grâce aux différents modes :

Nous avons 3 modes de gestion d'énergie : sleep mode(veille), normal mode(normal), forced mode (asynchrone +veille). Comme précisé dans nos rapports précédents on utilisera le mode forcé pour diminuer la consommation d'énergie.

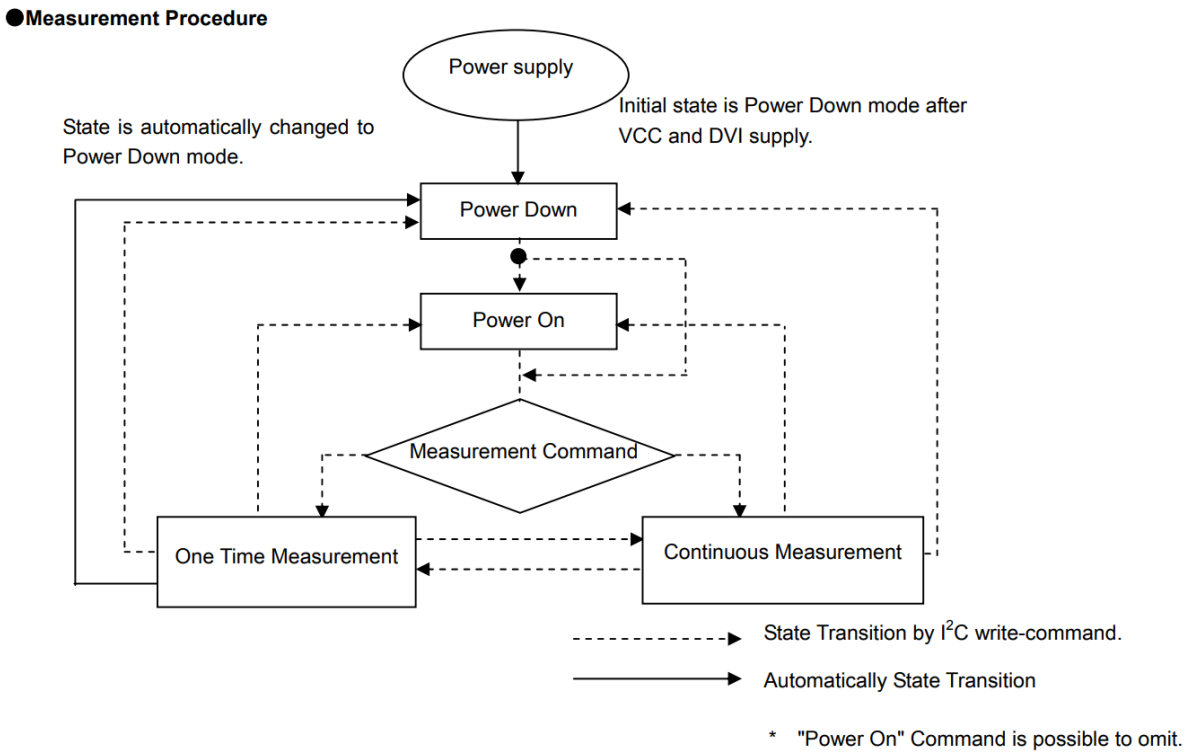
Les mesures seront faites en fonction de la précision voulue et de la consommation d'énergie du BME280.

b. Spécifications des mesures du capteur de pluie

Le capteur de pluie donne une valeur sur 10 bits donc de 2^{10} soit 0 à 1023.

c. Spécifications de mesure du capteur de luminosité BH1750 :

Schéma du processus de mesure :



Le capteur de luminosité renvoi une valeur en floattant non signée de 16 bits.

Extrait de datasheet:

H-resolution Mode2	Typ. 120ms.	0.5 lx

・ADC
AD converter for obtainment Digital 16bit data.
・Logic + I^2C Interface
Ambient Light Calculation and I^2C BUS Interface. It is including below register.
Data Register → This is for registration of Ambient Light Data. Initial Value is "0000_0000_0000_0000".

Wide range and High resolution. (1 - 65535 lx)

H-reslution mode2 : Illuminance per 1 count (lx / count) = 1 / 1.2 *(69 / X) / 2

d. Spécification de mesure du capteur de distance SFR02

Le capteur de distance donne un résultat de **16 bits non signées**.

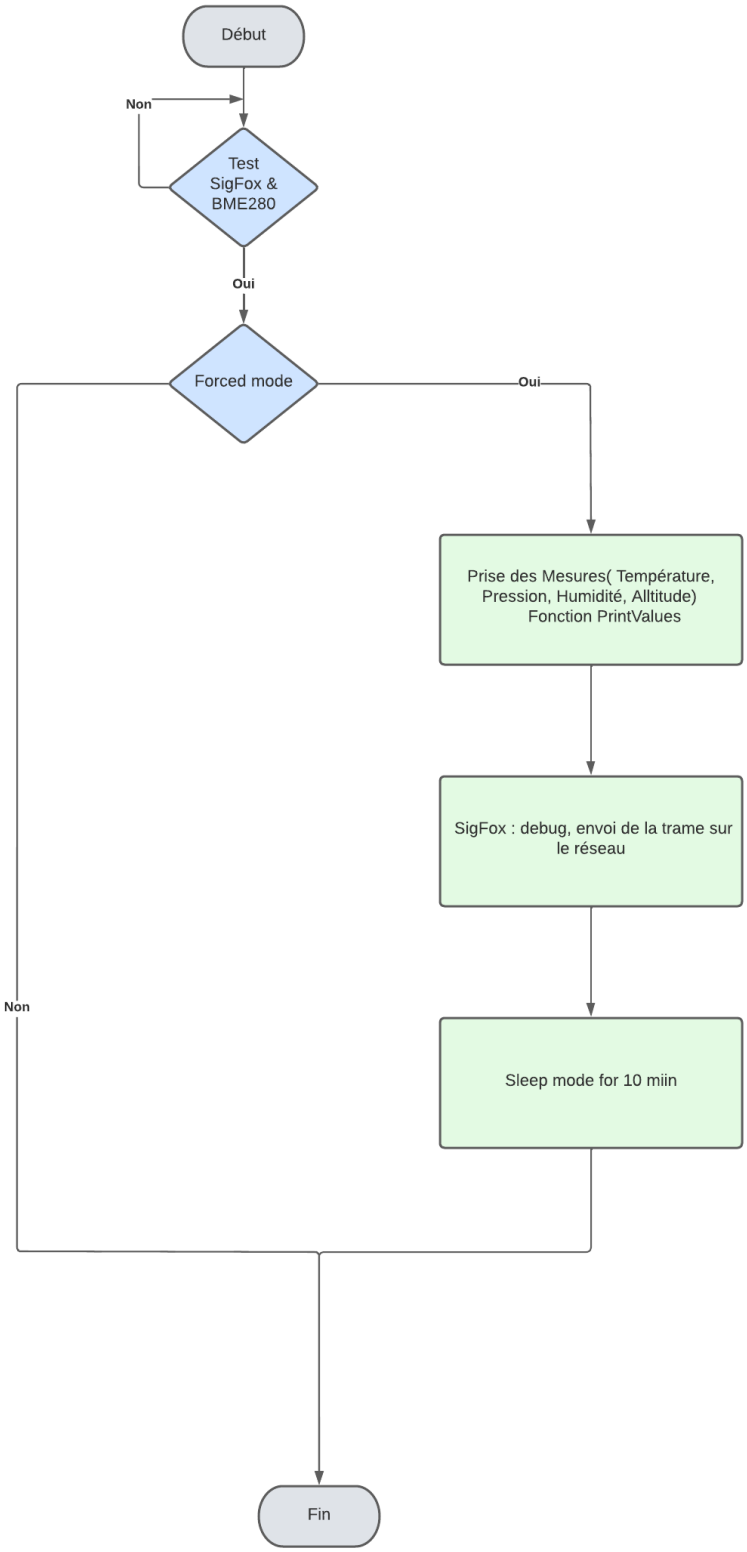
Extrait de la datasheet du SFR02 :

Locations, 2 and 3, are the 16bit unsigned result from the latest ranging - high byte first. The meaning of this value depends on the command used, and is either the range in inches, or the range in cm or the flight time in uS. A value of 0 indicates that no objects were detected. Do not initiate a ranging faster than every 65mS to give the previous burst time to fade away.

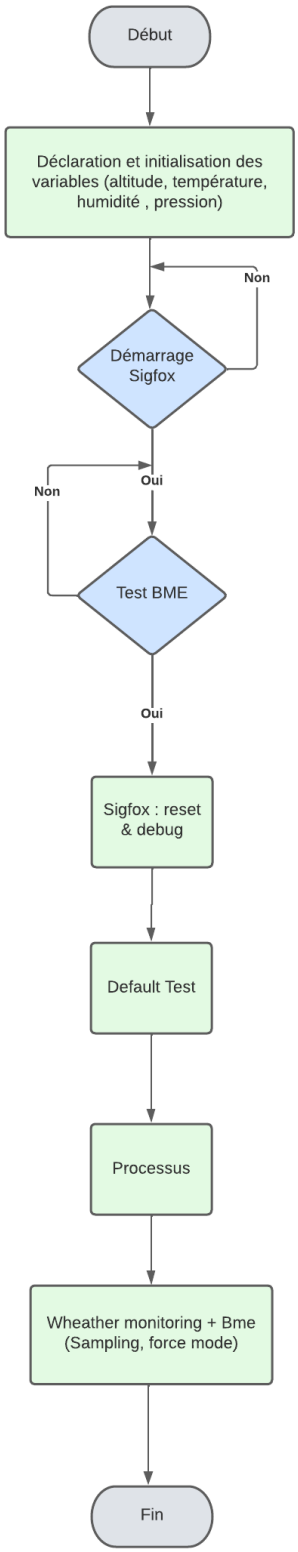
2	Range High Byte	N/A
3	Range Low Byte	N/A
81	0x51	Real Ranging Mode - Result in centimeters

Nous affichons les résultats de ce capteur en cm.

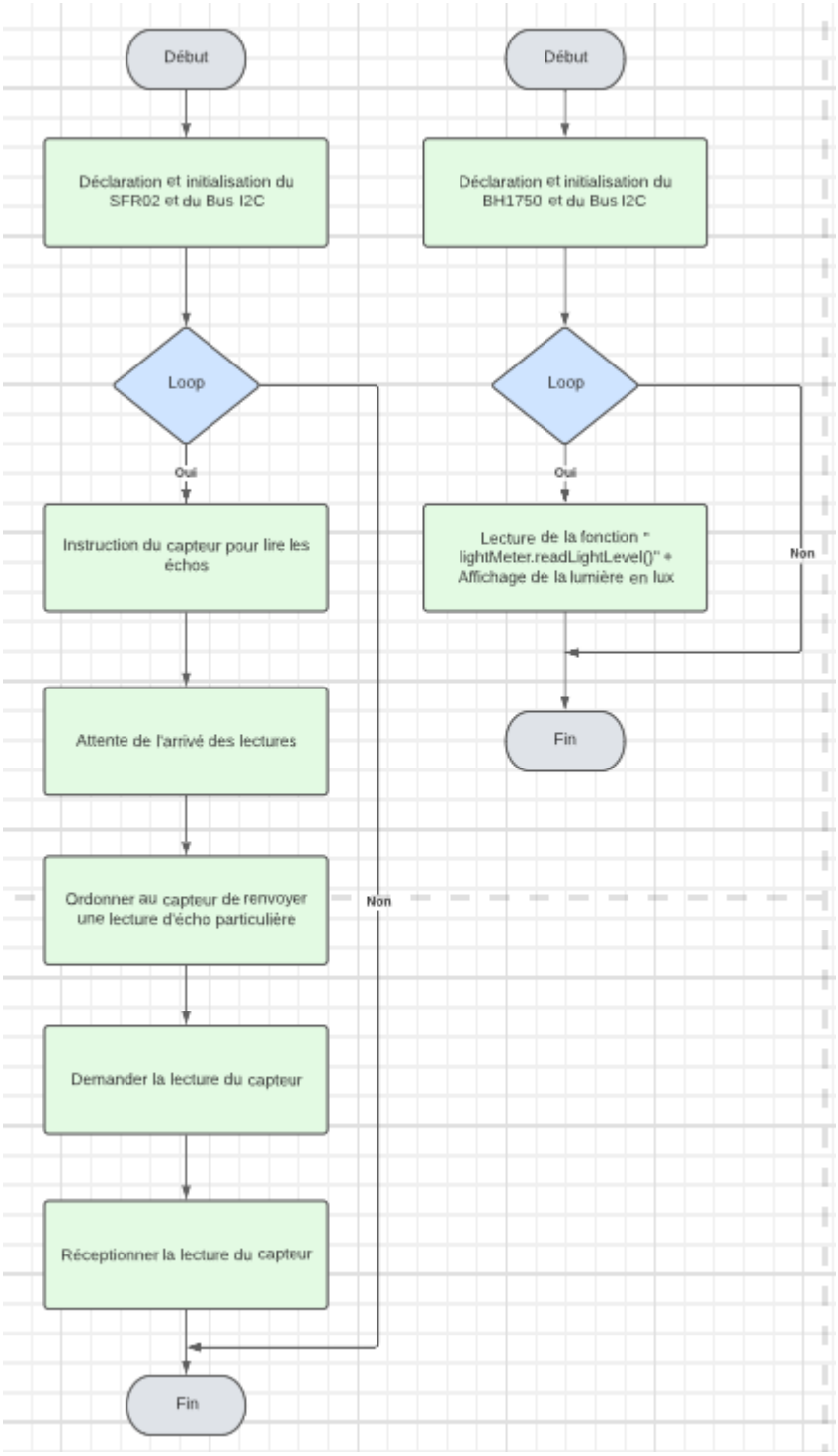
V. Organigrammes logiciels



Organigramme Sigfox



Organigramme BME280



Organigramme SFR02

Organigramme BH1750

VI. Expérimentations et tests :

BME280 :

Tout d'abord, nous avons testé le BME280 car c'est le composant principal de la station météo. Puis, nous avons ajouté le code adapté pour faire fonctionner le capteur de pluie.

Exemple de valeurs prise :

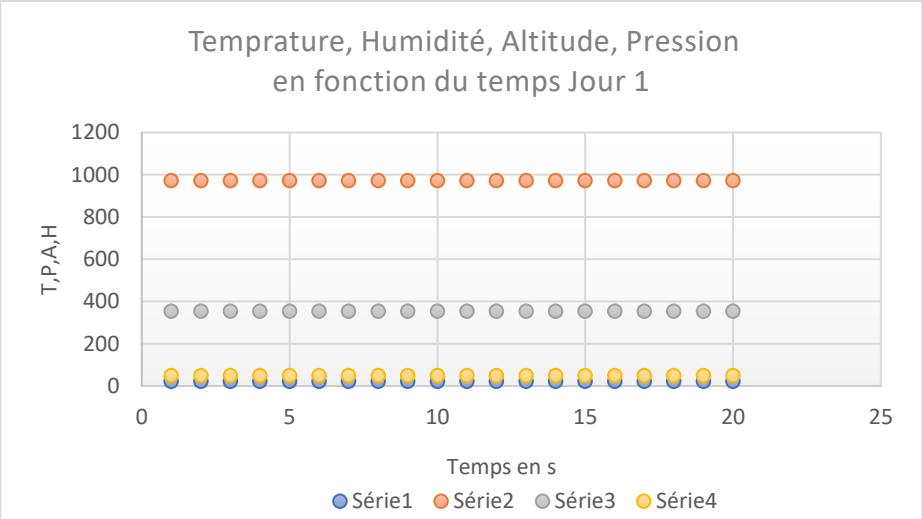
```
Temperature = 24.85 °C
Pressure = 978.44 hPa
Approx. Altitude = 358.58 m
Humidity = 44.35 %

ANALOG VALUE=408 OUTPUT= 0

=>il pleut c'est bien belfort ;)

Temperature = 24.85 °C
Pressure =
```

Nous avons donc résumé les données prises durant 3 jours différents et nous les avons mis dans un tableau grâce à Excel.

Test BME280 :

Jour 1 :

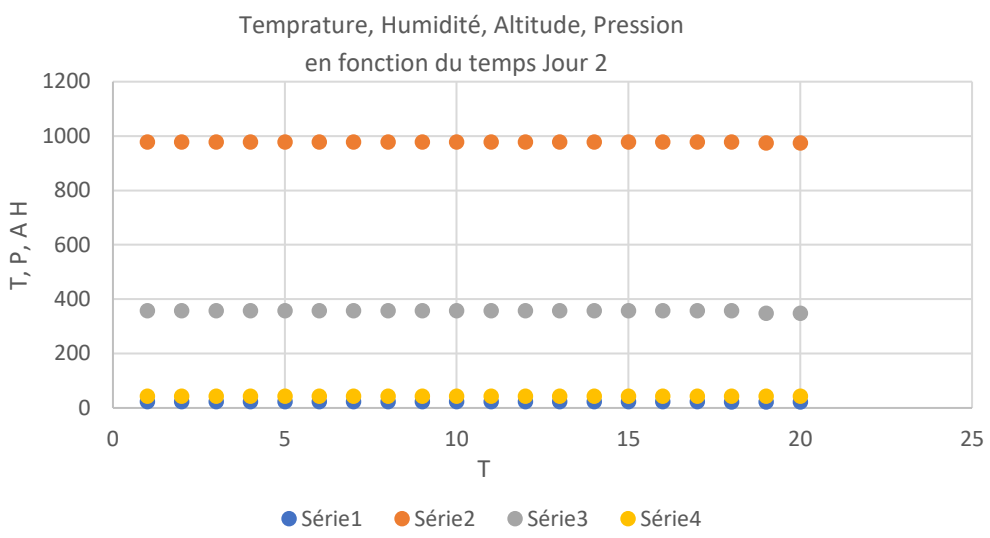
Temps (en s)	Température en °C	Pression en hPa	Altitude en m	Humidité en%
1	23	973	354,43	51
2	23	973	354,43	51
3	23	973	354,43	51
4	23	973	354,43	51
5	23	973	354,43	51
6	23	973	354,43	51
7	23	973	354,43	51
8	23	973	354,43	51
9	23	973	354,43	51
10	23	973	354,43	51
11	23	973	354,43	51
12	23	973	354,43	51
13	23	973	354,43	51
14	23	973	354,43	51
15	23	973	354,43	51
16	23	973	354,43	51
17	23	973	354,43	51
18	23	978	354,43	51
19	24	978	354,43	51
20	24	978	354,43	51



Temprature en °C, Humidité en %, Altitude en m, Pression en hPA

Jour 2 :

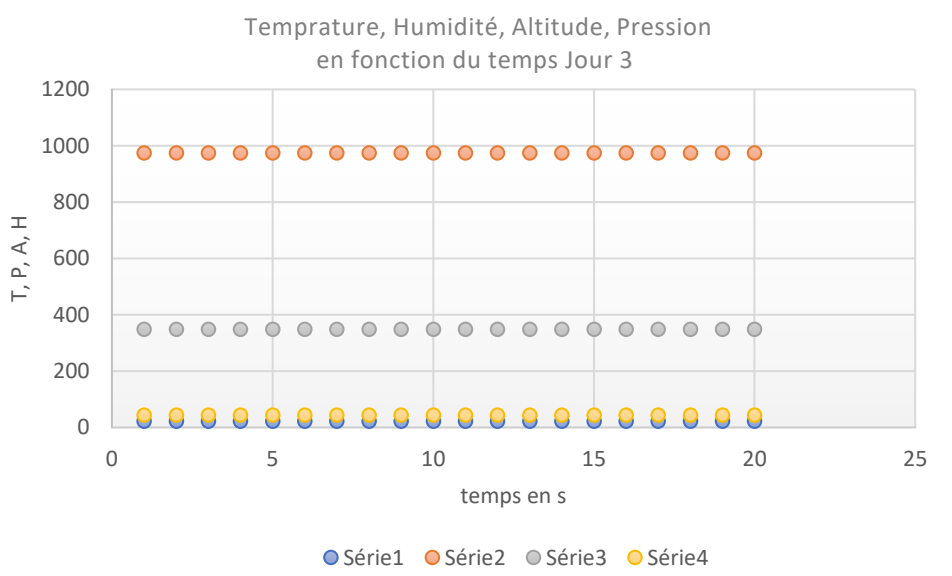
Temps (en s)	Température en °C	Pression en hPa	Altitude en m	Humidité en %
1	24	978	358	44
2	24	978	358	44
3	24	978	358	44
4	24	978	358	44
5	24	978	358	44
6	24	978	358	44
7	24	978	358	44
8	24	978	358	44
9	24	978	358	44
10	24	978	358	44
11	24	978	358	44
12	24	978	358	44
13	24	978	358	44
14	24	978	358	44
15	24	978	358	44
16	24	978	358	44
17	24	978	358	44
18	23	978	358	44
19	23	975	349	44
20	23	975	349	44



Temprature en °C, Humidité en %, Altitude en m, Pression en hPA

Jour 3 :

Temps (en s)	Température en °C	Pression en hPa	Altitude en m	Humidité en %
1	23	975	349	44
2	23	975	349	44
3	23	975	349	44
4	23	975	349	44
5	23	975	349	44
6	23	975	349	44
7	23	975	349	44
8	23	975	349	44
9	23	975	349	44
10	23	975	349	44
11	23	975	349	44
12	23	975	349	44
13	23	975	349	44
14	23	975	349	44
15	23	975	349	44
16	23	975	349	44
17	23	975	349	44
18	23	975	349	44
19	23	975	349	44
20	23	975	349	44



Temprature en °C, Humidité en %, Altitude en m, Pression en hPA

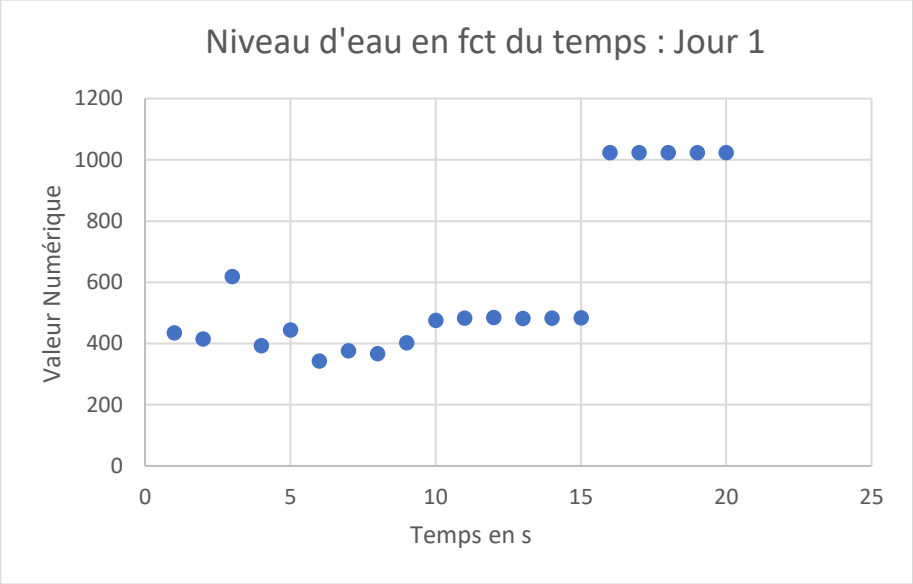
On peut voir qu'en ce qui concerne les données du BME280 les mesures prises sont les données attendues. L'intérêt de réaliser des mesures sur 3 jours différents est de voir les variations des températures, d'altitude de pression et d'humidité.

On remarque qu'au mois de mai la température était autour de 23°, l'altitude de 350m, l'humidité de 45%, et la pression de 975 hPa.

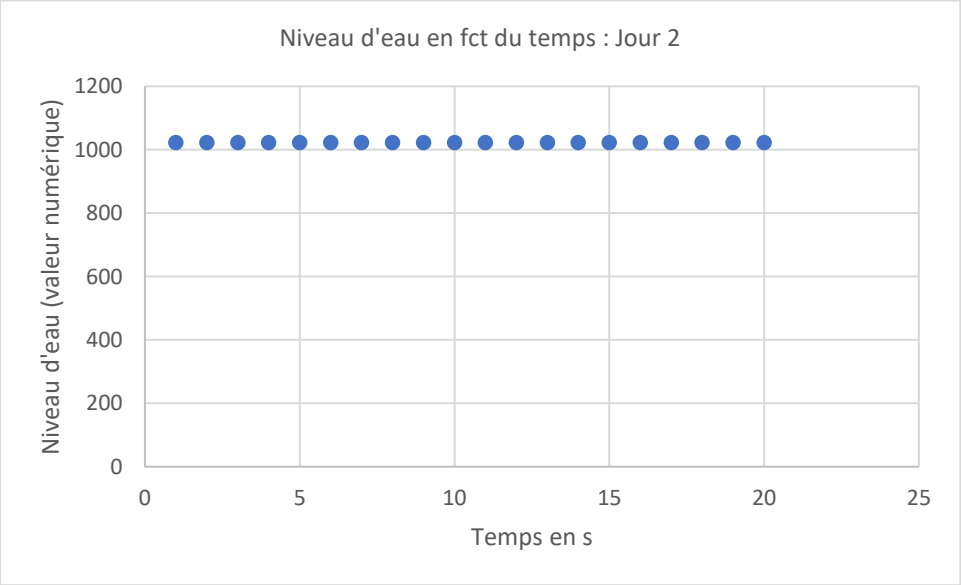
Test le capteur de pluie :

1023 correspond à une valeur où il n'y a pas du tout de pluie. Donc, plus il y avait de la pluie plus la valeur numérique baissait.

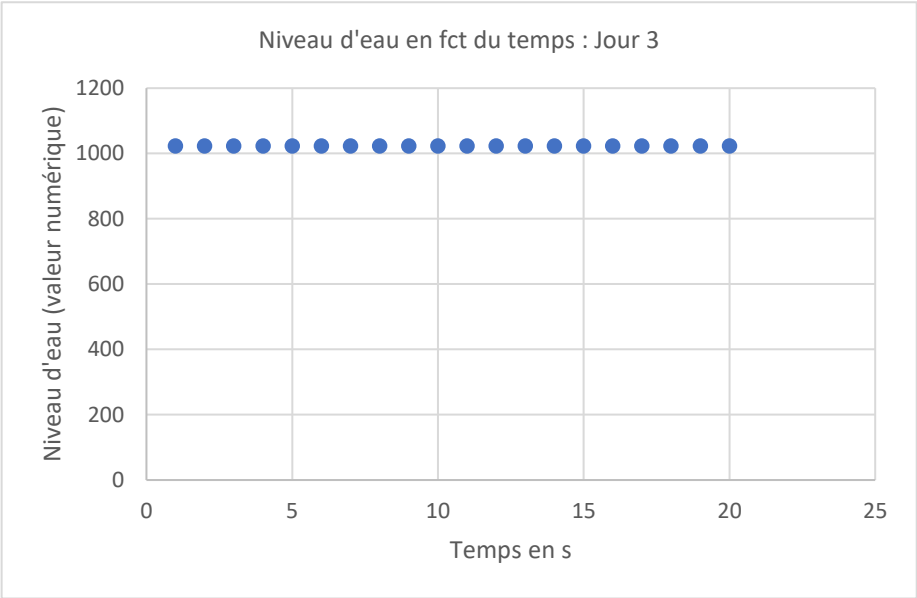
Temps (jour 1)	Pluie
1	435
2	415
3	619
4	393
5	444
6	343
7	376
8	367
9	402
10	475
11	483
12	485
13	482
14	483
15	484
16	1023
17	1023
18	1023
19	1023
20	1023



Temps (Jour 2)	Pluie
1	1023
2	1023
3	1023
4	1023
5	1023
6	1023
7	1023
8	1023
9	1023
10	1023
11	1023
12	1023
13	1023
14	1023
15	1023
16	1023
17	1023
18	1023
19	1023
20	1023



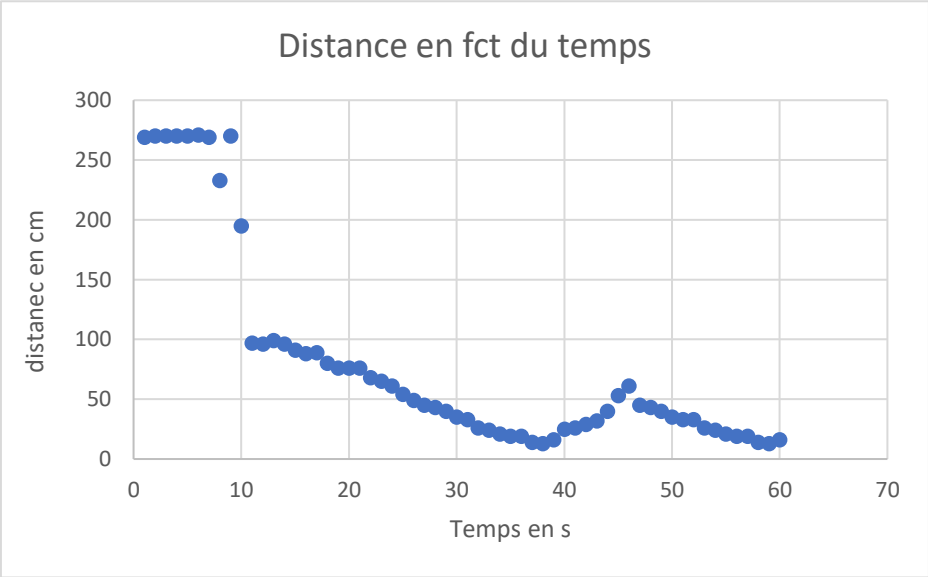
Temps (Jour 3)	Pluie
1	1023
2	1023
3	1023
4	1023
5	1023
6	1023
7	1023
8	1023
9	1023
10	1023
11	1023
12	1023
13	1023
14	1023
15	1023
16	1023
17	1023
18	1023
19	1023
20	1023



Pour les données du capteur de pluie, on remarque qu'au mois de mai le temps était plutôt sec.

Test du capteur de distance SFR02 :

Mesures	Distance en cm
1	269
2	270
3	270
4	270
5	270
6	271
7	269
8	233
9	270
10	195
11	97
12	96
13	99
14	96
15	91
16	88
17	89
18	80
19	76
20	76

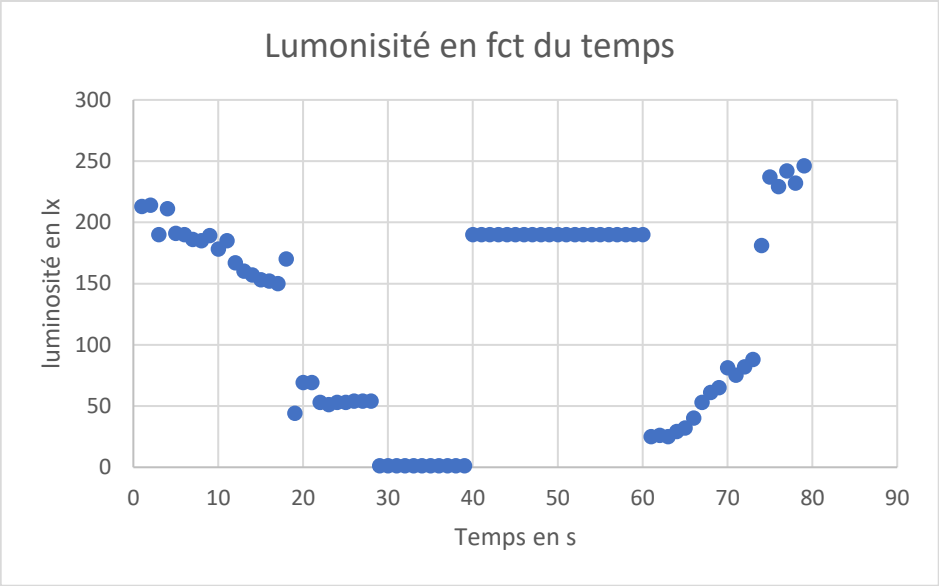


On a simulé la détection d'obstacle en plaçant un objet devant le capteur de distance. On peut voir la variation avec la courbe « distance en fct du temps ». 270cm correspond à la détection du plafond à

partir de la table. Les données du tableau ne sont pas exhaustives, en effet on a pris plus de 100 valeurs mais sur le graphique on affiche que 60 valeurs.

Test du capteur de luminosité BH1750 :

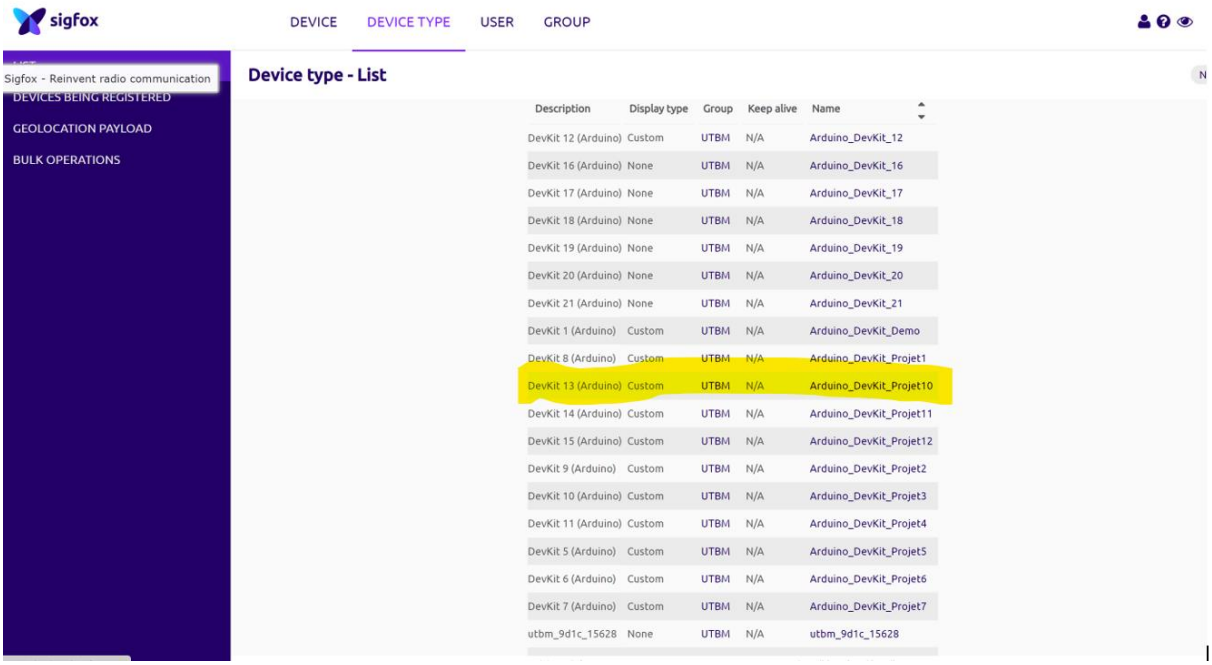
Mesures	Luminosité en lx
1	213
2	214
3	190
4	211
5	191
6	190
7	186
8	185
9	189
10	178
11	185
12	167
13	160
14	157
15	153
16	152
17	150
18	170
19	44
20	69

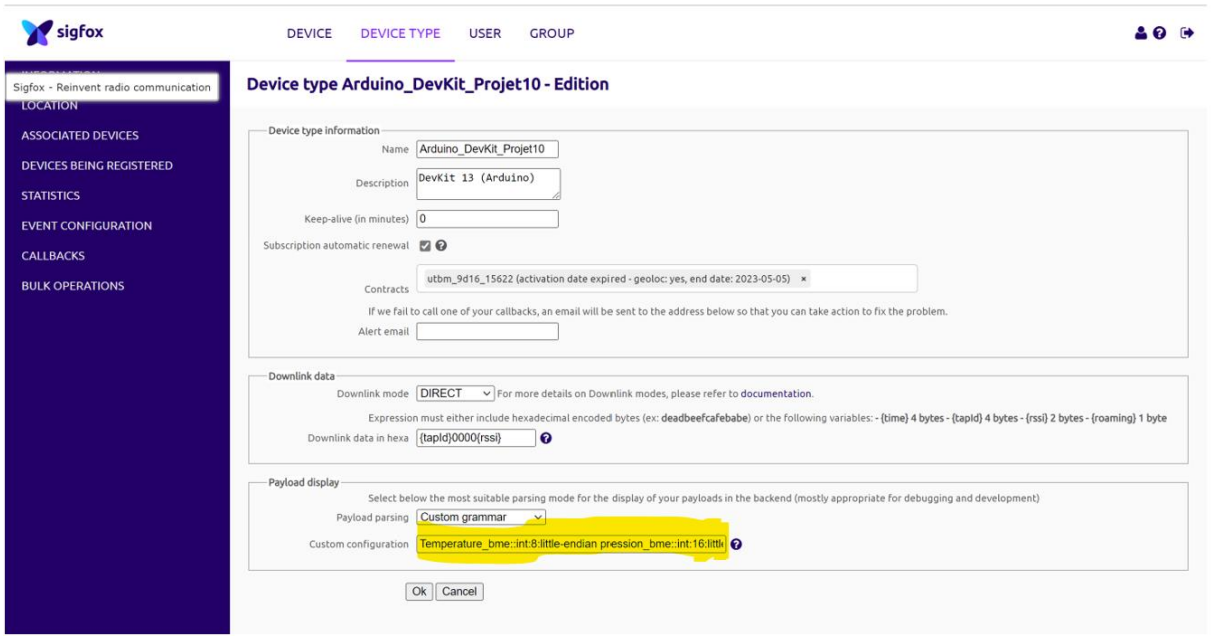


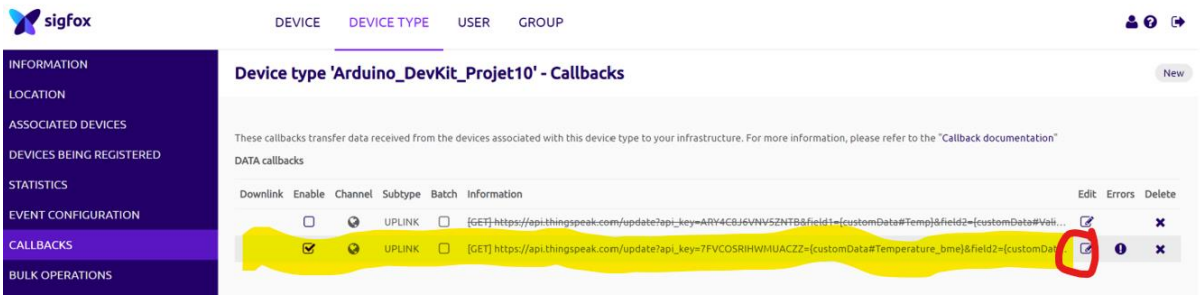
Pour le capteur de luminosité, on constate qu'à lumière du jour ambiante la valeur affichée est autour de 190 lx. Par la suite, on réduit l'éclairage pour simuler la nuit. Les données du tableau ne sont pas exhaustives, en effet on a pris plus de 100 valeurs mais sur le graphique on affiche que 80 valeurs. On alterne le jour et la nuit en recouvrant le capteur de luminosité.

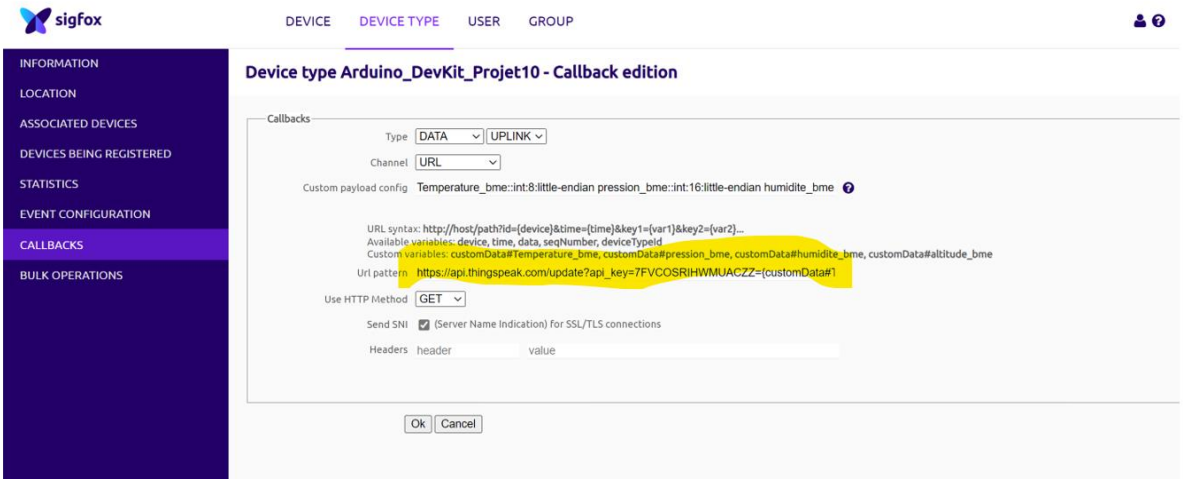
VII. Envoie des données sur Sigfox

Comme notre station météo se basait essentiellement sur le capteur BME280, on a envoyé les données de ce capteur via Sigfox pour cela on a suivi un protocole.

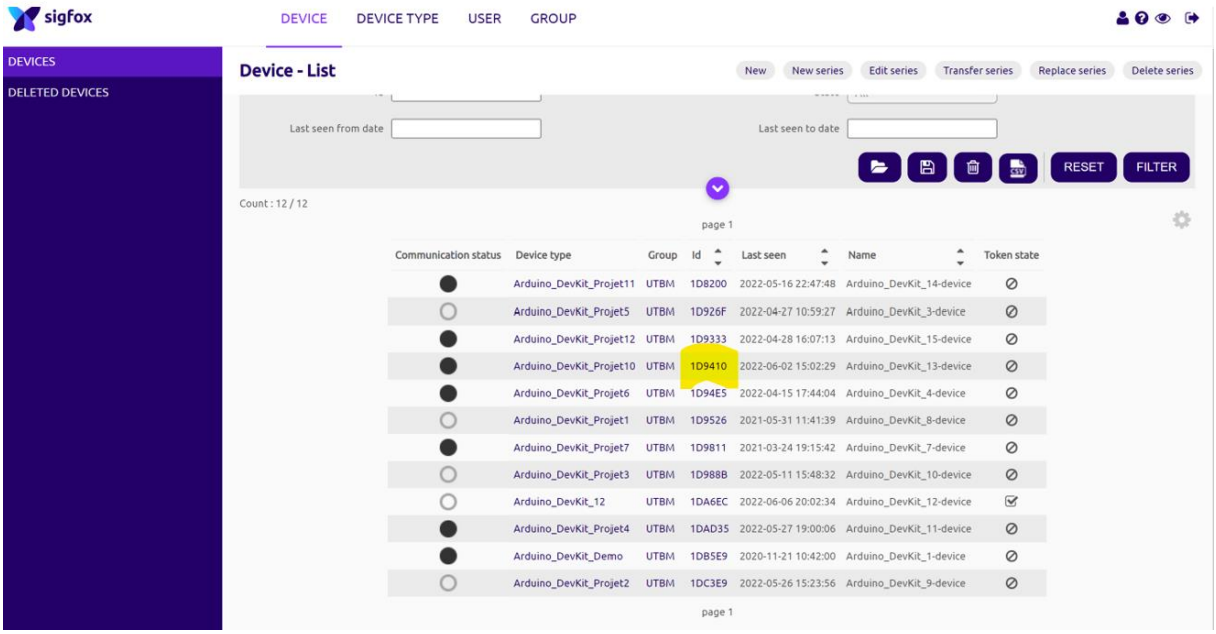


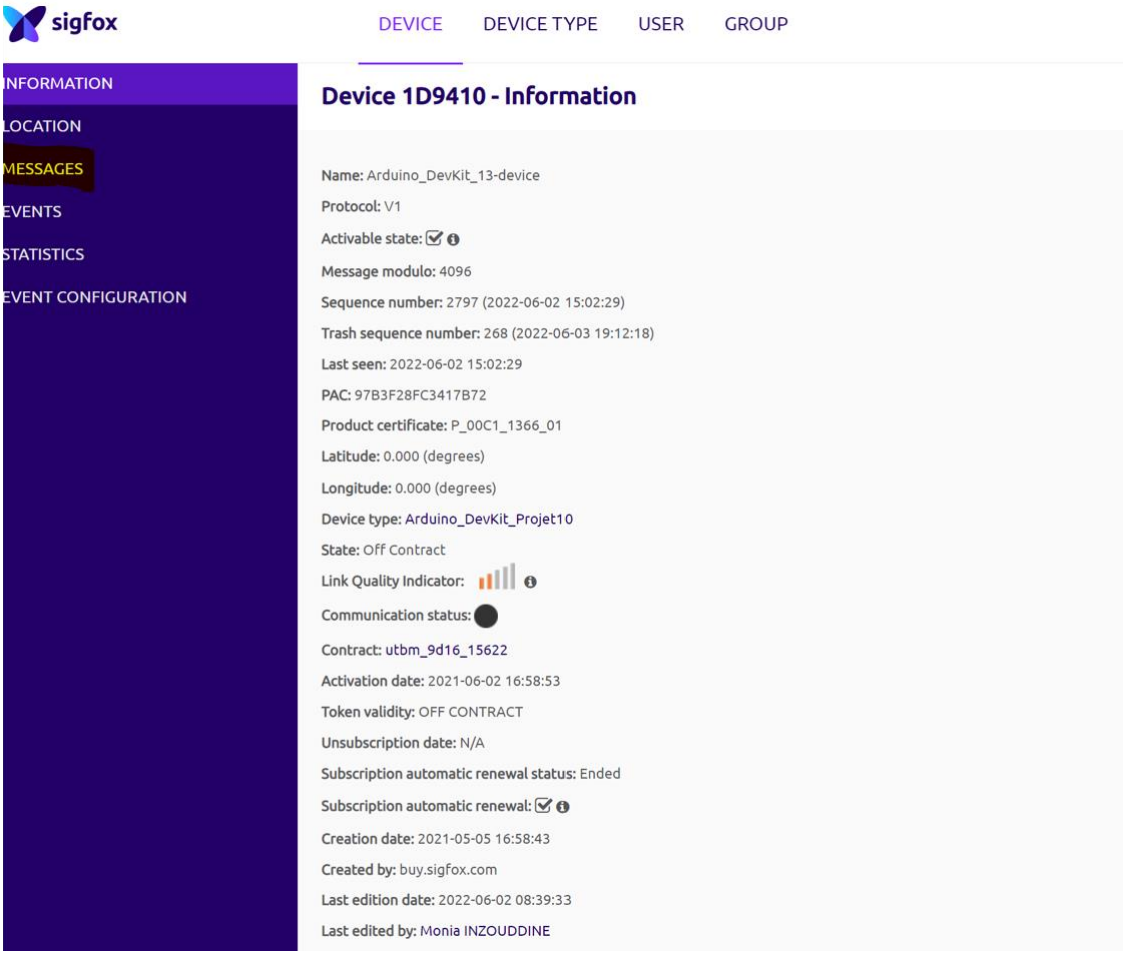




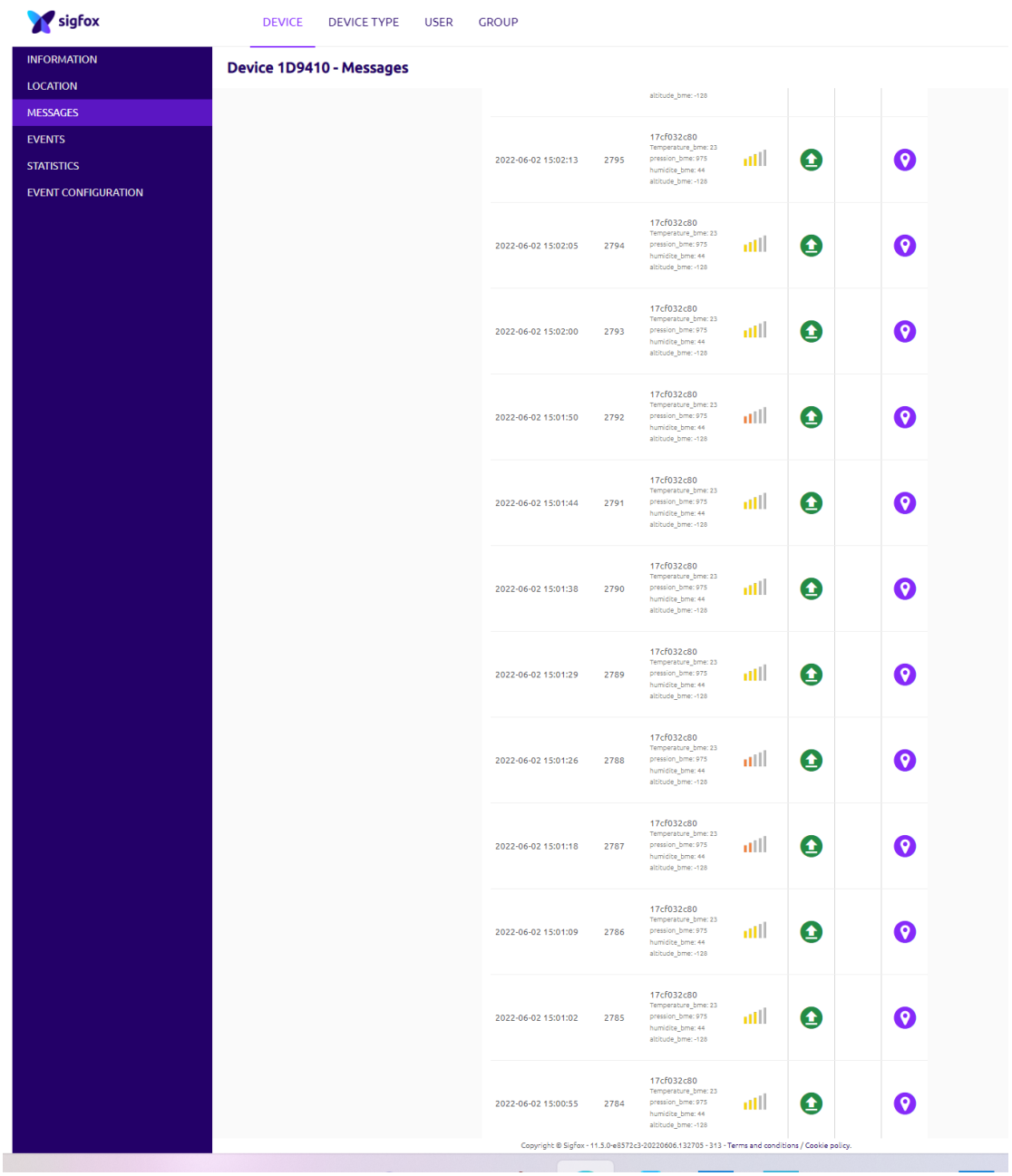


https://api.thingspeak.com/update?api_key=7FVCOSRIHWMUACZZ={customData#Temperature_bme}&field2={customData#pression_bme}&field3={customData#humidite_bme}&field4={customData#altitude_bme}





Après avoir suivi le protocole on obtenait les résultats suivants.

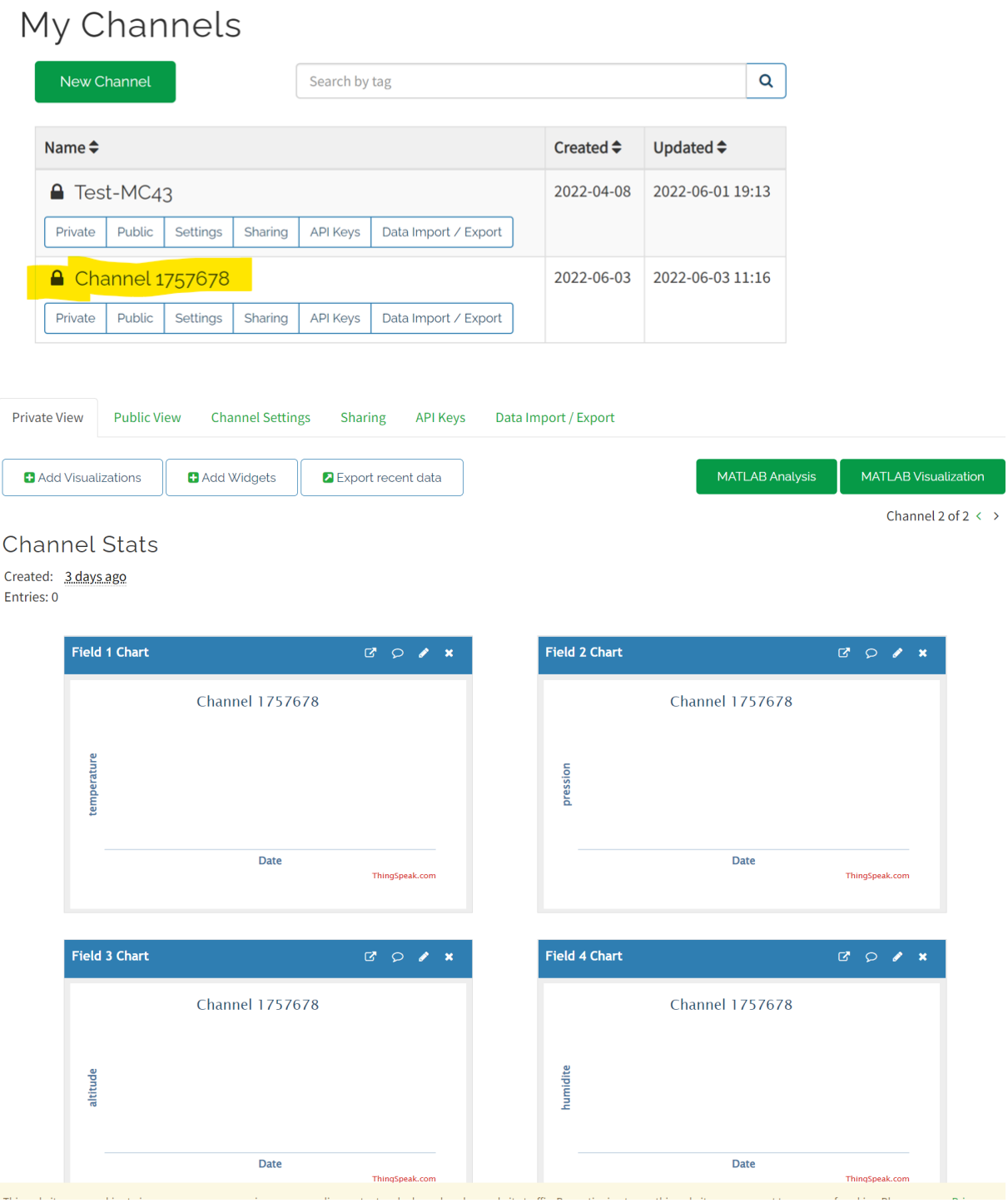

sigfox
DEVICE
DEVICE TYPE
USER
GROUP
INFORMATION
LOCATION
MESSAGES
EVENTS
STATISTICS
EVENT CONFIGURATION
Device 1D9410 - Messages
2022-06-02 15:02:13 2795 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:02:05 2794 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:02:00 2793 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:01:50 2792 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:01:44 2791 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:01:38 2790 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:01:29 2789 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:01:26 2788 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:01:18 2787 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:01:09 2786 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:01:02 2785 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:00:55 2784 17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
Copyright © Sigfox - 11.5.0-e8572c3-20220606.132705 - 313 - Terms and conditions / Cookie policy.

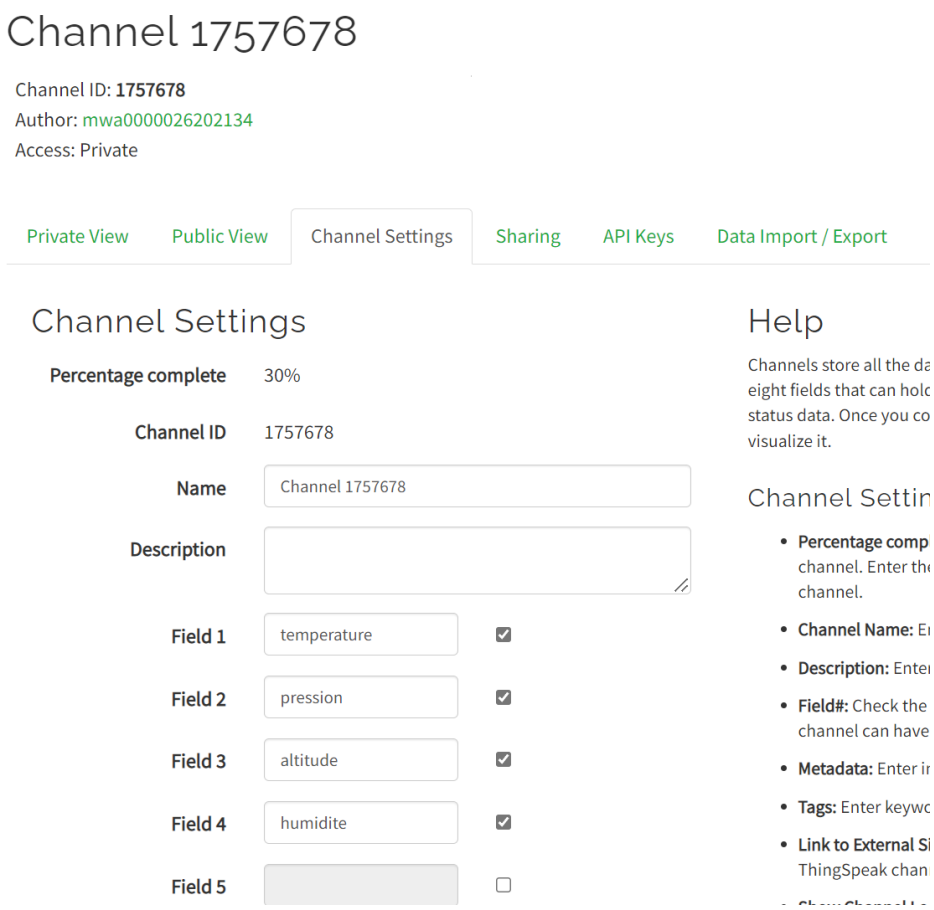
		humidite_bme: 44 altitude_bme: -128
2022-06-02 15:00:26	2780	17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:00:16	2779	17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:00:09	2778	17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 15:00:01	2777	17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 14:59:55	2776	17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 14:59:50	2775	17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 14:59:42	2774	17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 14:59:32	2773	17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 14:59:18	2771	17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
		Operator: SIGFOX_France Quality: Average
2022-06-02 14:59:11	2770	17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128
2022-06-02 14:59:04	2769	17cf032c80 Temperature_bme: 23 pression_bme: 975 humidite_bme: 44 altitude_bme: -128

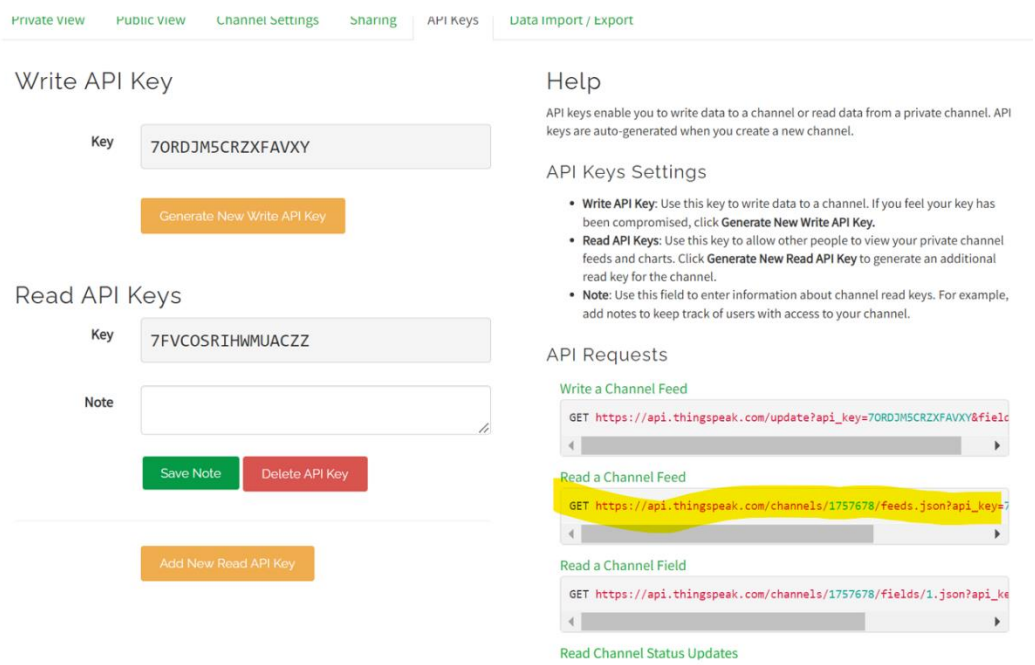
On voit ici que l'altitude n'a pas la bonne valeur, pour cela on a réalisé un débogage en comparant la valeur que récupérait le capteur et la valeur envoyée sur sigfox. Pour remédier à ce problème il faut ajouter un cast car on s'est rendu compte que le type des deux variables n'étaient pas identique.

VIII. ThingSpeak

Nous n'avons malheureusement pas pu envoyer les trames sur Thingspeak mais voici la configuration :

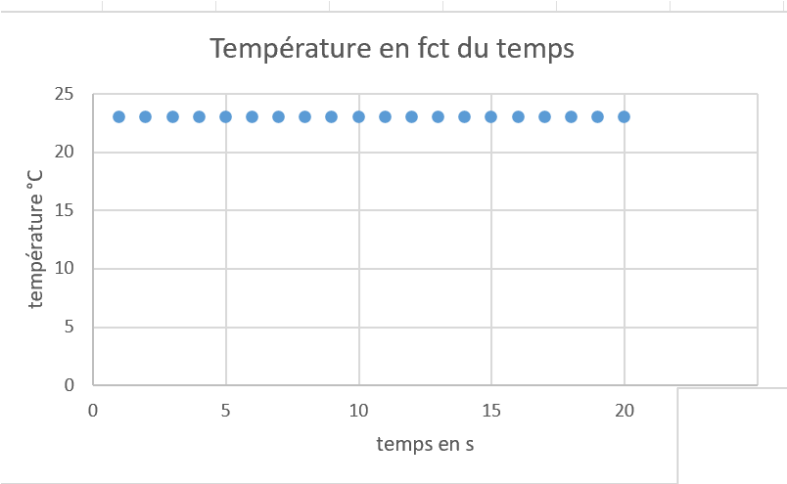


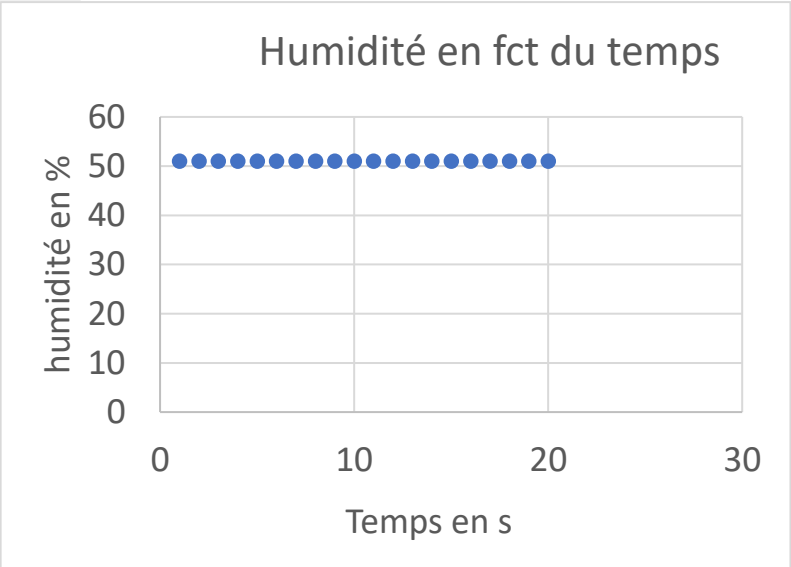


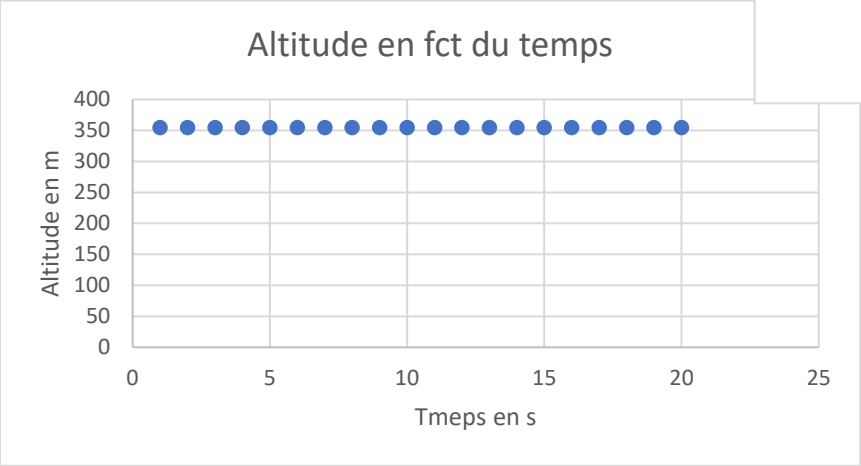


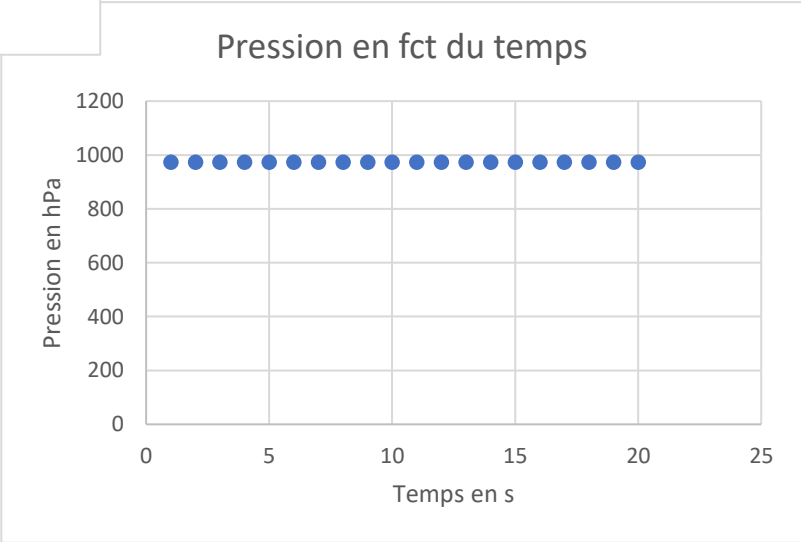
Pour la partie visualisation des données via Thinkspeak on a configuré les paramètres mais nous n'avons pas réussi à les retransmettre sur Thingspeak.

Pour avoir un aperçu des valeurs, il est préférable de reprendre les courbes Excel.


Température en fct du temps
température °C
25
20
15
10
5
0
0
5
10
15
20
temps en s


Humidité en fct du temps
humidité en %
60
50
40
30
20
10
0
0
10
20
30
Temps en s


Altitude en fct du temps
Altitude en m
400
350
300
250
200
150
100
50
0
0
5
10
15
20
25
Tmeps en s


Pression en fct du temps
Pression en hPa
1200
1000
800
600
400
200
0
0
5
10
15
20
25
Temps en s

IX. Conclusion :

Le projet **Mise en œuvre d'un objet connecté par réseau Sigfox** nous a permis de mieux combiner nos capacités afin de faire face aux divers problèmes rencontrés. Nous avons donc appris à mieux gérer les imprévus. Nous avons pu découvrir l'environnement Arduino, ainsi que l'envoi des données sur Sigfox. Pour cela les mises en œuvre utilisation de différents capteurs nous a permis d'approfondir nos connaissances en électronique et informatique industrielle, nous avons donc pu mettre en œuvre les connaissances de base sur l'utilisation des types, mais également les différentes fonctions sur Arduino. Pour notre part il était très intéressant de réaliser une station météo et de voir l'aboutissement du projet. En somme, nous sommes satisfaits de notre projet.

X. Annexe

Code pour BME280 et capteur de pluie

```
void setup() {
Serial.begin(115200);
Serial.println(F("BME280 station météo test la best"));
if (!SigFox.begin()) {
  Serial.println("Shield error or not present!");
  NVIC_SystemReset(); // rebooting
  while(1);
}


if (! bme.begin(0x76, &Wire)) {
  Serial.println("Could not find a valid BME280 sensor, check wiring!");
  while (1);
  }

SigFox.reset();
delay(1000);
SigFox.debug(); // Enable debug prints and LED indication if we are testing
SigFox.end(); //Send module to standby until we need to send a message


Serial.println("-- Default Test --");
//Serial.println("normal mode, 16x oversampling for all, filter off,");
//Serial.println("0.5ms standby period");
//delayTime = 5000;

  // weather monitoring
  Serial.println("-- Weather Station Scenario --");
  Serial.println("forced mode, 1x temperature / 1x humidity / 1x pressure oversampling,");
  Serial.println("filter off");
  bme.setSampling(Adafruit_BME280::MODE_FORCED,
                  Adafruit_BME280::SAMPLING_X1, // temperature
                  Adafruit_BME280::SAMPLING_X1, // pressure
                  Adafruit_BME280::SAMPLING_X1, // humidity
                  Adafruit_BME280::FILTER_OFF   );

  // suggested rate is 1/60Hz (1m)
  //delayTime = 60000; // in milliseconds
  Serial.println();
// Configure the sensors and populate the status field ...
```

```
void loop() {
  // Every 5 min minutes, read all the sensors and send them
  // Let's try to optimize the data format

  SigFox.begin(); // Start the module
  delay(100); // Wait at least 30ms after first configuration (100ms before)
  SigFox.status(); // Clears all pending interrupts
  SigfoxMessage msg;
 // bme.takeForcedMeasurement(); // has no effect in normal mode
  msg.temperature = bme.readTemperature();
  msg.pression = ((float)(bme.readPressure() / 100.0F));
  msg.humidite = bme.readHumidity();
  msg.altitude = (int)bme.readAltitude(pressionnivmer);


  printValues(msg.temperature, msg.pression, msg.humidite, msg.altitude);

  // affichage pour debug
  //Serial.println("MKRFox1200");

  // Envoi de la trame sur le reseau sigfox
  SigFox.beginPacket();
  SigFox.write((uint8_t*)&msg, sizeof(msg));
  SigFox.endPacket();
  SigFox.end();
  //delay(10000);
  // READ THE ANALOG INPUT ON PIN AO
  int ANALOG_VALUE = analogRead(A0);
  // READ THE-DIGITAL INPUT ON PIN 8
  int OUTPUT_VALUE = digitalRead(8);

  //PRINT OUT THE VALUE READING
  Serial.print("ANALOG VALUE=");
  Serial.print(ANALOG_VALUE);
  Serial.print("");
  Serial.print(" OUTPUT= ");
  Serial.println(OUTPUT_VALUE);
  Serial.println(" ");
  if(analogRead(0)<350) Serial.println("=>Il pleut énormément.");
  else if(analogRead(0)<500) Serial.println("=>Il pleut modéremment.");
  else Serial.println("=> Il n'y a pas de pluie.");
  Serial.println(" ");
  //Sleep for min minutes
  //LowPower.deepSleep(((min*60)-t_mesure) * 1000); // Durée de veille en ms



}
void printValues(uint8_t temperature, uint16_t pression, uint8_t humidite, uint8_t altitude) {
    Serial.print("Temperature = ");
    Serial.print(temperature);
    Serial.println(" °C");

    Serial.print("Pressure = ");
    Serial.print(pression);
    Serial.println(" hPa");

    Serial.print("Approx. Altitude = ");
    //Serial.print(bme.readAltitude(SEALEVELPRESSURE_HPA));
    //Serial.print(altitude);
    Serial.print(bme.readAltitude(pressionnivmer));

    Serial.println(" m");

    Serial.print("Humidity = ");
    Serial.print(humidite);
    Serial.println(" %");

    Serial.println();
}
```

Code SFR02 :

```
// I2C SRF10 or SRF08 Devantech Ultrasonic Ranger Finder
// by Nicholas Zambetti <http://www.zambetti.com>
// and James Tichenor <http://www.jamestichenor.net>

// Demonstrates use of the Wire library reading data from the
// Devantech Utrasonic Rangers SFR08 and SFR10

// Created 29 April 2006

// This example code is in the public domain.

#include <Wire.h>

void setup() {

  Wire.begin();                // join i2c bus (address optional for master)
  Serial.begin(115200);          // start serial communication at 9600bps
}

int reading = 0;

void loop() {

  // step 1: instruct sensor to read echoes
      Wire.beginTransmission(112); // transmit to device #112 (0x70)
      // the address specified in the datasheet is 224 (0xE0)
      // but i2c adressing uses the high 7 bits so it's 112
      Wire.write(byte(0x00));      // sets register pointer to the command register (0x00)
      Wire.write(byte(0x51));      // command sensor to measure in "inches" (0x50)
      // use 0x51 for centimeters
      // use 0x52 for ping microseconds
      Wire.endTransmission();      // stop

  // step 2: wait for readings to happen
      delay(1000);                   // datasheet suggests at least 65 milliseconds
  // step 3: instruct sensor to return a particular echo reading
      Wire.beginTransmission(112); // transmit to device #112
      Wire.write(byte(0x02));      // sets register pointer to echo #1 register (0x02)
      Wire.endTransmission();      // stop transmitting
  // step 4: request reading from sensor
      Wire.requestFrom(112, 2);    // request 2 bytes from slave device #112
  // step 5: receive reading from sensor
      if (2 <= Wire.available()) { // if two bytes were received

    reading = Wire.read();  // receive high byte (overwrites previous reading)
    reading = reading << 8;    // shift high byte to be high 8 bits

    reading |= Wire.read(); // receive low byte as lower 8 bits
    Serial.print("La distance de l'obstacle mesuré est de : ");
    Serial.print(reading);   // print the reading
    Serial.println(" cm");

  }

  delay(250);                  // wait a bit since people have to read the output :)
}
```

Code pour luminosité

```
/*
  Example of BH1750 library usage. This example initialises the BH1750 object using the defau
*/

#include <Wire.h>
#include <BH1750.h>

BH1750 lightMeter;

void setup(){
  Serial.begin(115200);

  // Initialize the I2C bus (BH1750 library doesn't do this automatically)
  Wire.begin();
  // On esp8266 you can select SCL and SDA pins using Wire.begin(D4, D3);
  // For Wemos / Lolin D1 Mini Pro and the Ambient Light shield use Wire.begin(D2, D1);
  lightMeter.begin();
  Serial.println(F("BH1750 Test begin"));
}

void loop() {
  float lux = lightMeter.readLightLevel();
  Serial.print("Light: ");
  Serial.print(lux);
  Serial.println(" lx");
  delay(1000);
}
```

Quelques résultats du moniteur série :

```
Temperature = 24.20 °C
Pressure = 978.74 hPa
Approx. Altitude = 396.88 m
Humidity = 46.90 %

Temperature = 24.24 °C
Pressure = 978.78 hPa
Approx. Altitude = 396.47 m
Humidity = 47.31 %

Temperature = 24.24 °C
Pressure = 978.76 hPa
Approx. Altitude = 396.71 m
Humidity = 46.78 %
```